



DATA · ENERGY · INTELLIGENCE

Full-Stack Build Brief · For Claude Code

PetroSense Platform

The complete build plan — Frontend, Backend, Database, and API integration — with a ready-to-paste Claude Code prompt for every stage. React frontend · Django backend · PostgreSQL.

React + Vite

Django + DRF

PostgreSQL

Admin + Engineer roles

TEAM: Youssef Hamada · Zoher Mohamed · Ammar Yasser · Nady Rabie
Supervisor: Noha Hussen · ExxonMobil × SUT — Summer Program 2026

How to Use This Brief



This document is written to be fed to **Claude Code**. Build in the order given — **Frontend first, then Backend, then Database, then API integration**. Paste each stage's grey prompt block one at a time; review the result, then continue. Never paste all stages at once.

Golden rule for every prompt: keep scope surgical. Ask Claude Code to build exactly what the stage says, reuse existing components, and never redesign earlier work unless the stage asks for it.

The Stack

Frontend	React + Vite + TypeScript · Tailwind · Framer Motion · Recharts	Fast, smooth, animation-rich dark UI.
Backend	Django + Django REST Framework	Batteries-included, admin-friendly, fast to build.
Database	PostgreSQL	Reliable, great for time-series sensor data.
Auth	JWT (SimpleJWT) + roles	Admin vs Engineer access control.
ML	scikit-learn Random Forest	Supervised model + feature importance = Explainable AI.



Who Does What

PetroSense has two roles. Every screen and API respects them.

ENGINEER (STANDARD USER)

- Log in to the dashboard.
- See the field Overview — wells online, active alerts, average health.
- Open any well and watch its 4 live sensors (pressure, temperature, vibration, flow).
- Read Alerts with the explainable reason behind each one.
- Explore Analytics — trends and anomaly frequency.
- Adjust their own Settings (profile, thresholds, notifications).

ADMIN (FULL CONTROL)

- Everything an Engineer can do, plus:
- **Manage users** — add, delete, and set each user's role (admin / engineer).
- **Manage wells** — add a new well or remove one directly from the interface.
- See the admin-only navigation items (hidden entirely from engineers).
- Admin routes are enforced on the backend too — an engineer token is rejected (403).

Note on Settings: the "Connection / API" section is an intentional placeholder — shown but disabled and badged "Coming soon" — so the UI is ready for a future feature without pretending it works yet.

PART 1

Part 1 — Frontend (React)

Build the frontend first, entirely on mock data, so you can see and click the whole product before any backend exists. Every data call already goes through one file (`/src/services/api.ts`) — later you swap only that file.

Build the frontend first, entirely on mock data, so you can see and click the whole product before any backend exists. Every data call already goes through one file (`/src/services/api.ts`) — later you swap only that file.

1

STAGE 1

Foundation & Setup

Project scaffold, design tokens, and the API-ready mock layer with admin functions.

PASTE THIS INTO CLAUDE CODE

Set up a new React frontend for "PetroSense", an AI-driven oil-well monitoring platform.

STACK: React + Vite + TypeScript, Tailwind CSS, React Router, Framer Motion, Recharts, TanStack Query, Axios, Zustand, lucide-react.

DO IN THIS STAGE ONLY:

1. Scaffold Vite + React + TS and install all dependencies above.
2. Configure Tailwind with these EXACT dark-theme tokens:
bg #141B22 · surface #1E2831 · border #2F3D49 · primary/flame #E8821E
accent/amber #F2A93B · teal #2A8F9C · teal-light #4BB8C4 · text #E8EEF2
muted #8A99A6 · normal #3BA55D · warning #F2A93B · critical #E4572E
radius: cards 14px, inputs 10px. Font: Inter (UI), tabular/mono for numbers.
3. Folders: `/src/components` `/src/pages` `/src/layouts` `/src/services`
`/src/store` `/src/types` `/src/lib`
4. Build `/src/services/api.ts` as the ONLY data source. Mock now, real later.
Functions: `getWells`, `getWell(id)`, `getAlerts`, `getKpis`, `getSensorHistory(id)`,
`login`, `getCurrentUser`, `getUsers`, `createUser`, `deleteUser`, `createWell`, `deleteWell`.
Mock 5 wells (Well_001..005) with 4 sensors: `pressure_bar`, `temperature_c`,
`vibration_mm_s`, `flow_rate_m3_day`, plus `is_anomaly`. Mark clearly where Axios
calls will replace mocks.
5. Types in `/src/types`: `Well`, `SensorReading`, `Alert`, `Kpi`, `User` (`role: 'admin' | 'engineer'`).
6. Router routes: `/`, `/login`, `/app`, `/app/well/:id`, `/app/alerts`,
`/app/analytics`, `/app/settings`, `/app/admin/users`, `/app/admin/wells`.
7. A Zustand auth store holding `{currentUser, role, token}`. Respect `prefers-reduced-motion`.

Keep it minimal: a running app, tokens, and the mock API layer. Show the folder tree.

2

STAGE 2

Landing + Login

The public marketing page and a login that reads the user's role.

PASTE THIS INTO CLAUDE CODE

Build the PUBLIC pages for PetroSense: Landing (/) and Login (/login).
Existing tokens, Tailwind, Framer Motion, dark theme.

LANDING (/):

1. Sticky slim navbar: logo left; links (Features, How it works, Login) right.
2. HERO: headline "AI-Driven Smart Oil-Well Monitoring", subheading on detecting problems early + predicting failures, primary CTA "Open Dashboard" -> /login. Animated gradient/particle background in flame+teal. Staggered fade+slide-up.
3. FOUR capability cards (stagger on scroll): Early Detection, Predictive Maintenance, Real-Time Monitoring, Explainable AI. lucide icon + title + line.
4. HOW IT WORKS: 4-step flow Collect -> Analyze -> Predict -> Recommend, animated arrows.
5. Closing CTA band + minimal footer (ExxonMobil x SUT 2026).

LOGIN (/login):

- Centered card, logo, email + password, "Sign in" -> calls api.login() then routes to /app. Read role from the returned user into the Zustand store. UI polished, subtle entrance animation. (Still mock auth – api.login returns a mock user+token.)

Framer Motion entrance + scroll reveals, 150-350ms ease-out, respect reduced-motion.
Do NOT build the dashboard yet. Reuse tokens; don't restyle globals.

3

STAGE 3

Dashboard Shell + Overview

The reusable layout with role-aware nav, then the field overview.

PASTE THIS INTO CLAUDE CODE

Build the DASHBOARD SHELL and the OVERVIEW screen. Dark theme, existing tokens.

PART A - APP SHELL (layout for all /app screens):

- Fixed left SIDEBAR: logo; nav with lucide icons (Overview, Wells, Alerts, Analytics, Settings). Show ADMIN-ONLY nav items (Manage Users, Manage Wells) ONLY when the Zustand store role === 'admin'. Active highlight; collapsible.
- TOP BAR: well selector (Well_001..005), device/connection status dot, user menu showing name + role, with Logout.
- Content area renders routed screen with Framer Motion AnimatePresence (fade+slide-up).
- Add a <RequireRole role="admin"> wrapper component that redirects non-admins away from /app/admin/* routes.

PART B - OVERVIEW (/app):

- KPI cards with count-up: Wells Online, Active Alerts, Avg Health, Anomalies Today.
- WELL STATUS GRID: card per well (name, Normal/Warning/Critical badge with animated dot, Critical pulses, mini sparkline). Click -> /app/well/:id. Staggered entrance.
- RECENT ALERTS feed on the side.

All data from api.ts (mock). Loading skeletons. Responsive. Surgical scope.

4

STAGE 4

Well Detail + Alerts

Live sensor charts and the explainable alerts screen.

PASTE THIS INTO CLAUDE CODE

Build WELL DETAIL (/app/well/:id) and ALERTS (/app/alerts).
Dark theme, tokens, Recharts + Framer Motion.

WELL DETAIL:

- Header: well name, overall status badge, last-updated.
- FOUR live line charts (one per sensor: pressure, temperature, vibration, flow), each in a card, colored line, smooth transitions, current-value readout with count-up.
- Simulate live updates by appending mock points on an interval from api.ts.
- Per-signal status strip: Normal / Warning / Critical.

ALERTS:

- Filterable list (by severity + by well). Each row: severity color, well, sensor, timestamp, and a prominent EXPLAINABLE REASON line (e.g. "Vibration 6.5 mm/s – 62% above normal baseline"). This is the differentiator.
- Click a row to expand: which sensor drove it, reading vs baseline. Smooth expand.

All data via api.ts. Loading + empty states. Reuse components; surgical.

5

STAGE 5

Analytics + Settings + Admin

Analytics, Settings (with the disabled API placeholder), and the admin Users & Wells screens.

PASTE THIS INTO CLAUDE CODE

Build ANALYTICS, SETTINGS, then the ADMIN screens.
Dark theme, tokens, Recharts + Framer Motion.

ANALYTICS (/app/analytics):

- Anomaly frequency over time (bar/area), sensor comparison across wells, normal-vs-anomaly breakdown. Date/well filter bar. Animate on load.

SETTINGS (/app/settings):

- Profile section, alert thresholds per sensor, notification preferences.
- A "Connection / API" section that is DISABLED and clearly badged "Coming soon" (greyed out, not clickable) – this is an intentional placeholder.

ADMIN – USERS (/app/admin/users) [admin only]:

- Table of users (name, email, role, created). Add-user modal (name, email, role: admin/engineer, password). Delete-user with confirm dialog.
- Calls api.getUsers / api.createUser / api.deleteUser.

ADMIN – WELLS (/app/admin/wells) [admin only]:

- Table of wells (name, location, status, created). Add-well modal (name, location). Delete-well with confirm. Calls api.getWells / api.createWell / api.deleteWell.

Wrap both admin routes in <RequireRole role="admin">. Loading + empty states, optimistic UI on add/delete, toasts on success/error. Reuse components; surgical.

Transitions, loading states, responsiveness, and a final role-access check.

PASTE THIS INTO CLAUDE CODE

Polish pass across the WHOLE PetroSense app. No redesigns – refine only.

1. Consistent Framer Motion page transitions on every route.
2. Loading skeletons + graceful empty states on every data view.
3. Full responsiveness: sidebar collapses, grids reflow, charts resize.
4. Micro-interactions: hover lifts, animated status dots, button feedback, toasts.
5. Respect prefers-reduced-motion; visible keyboard focus states; basic ally labels.
6. Final visual QA vs tokens (spacing, radius, colors). Confirm admin nav/routes are hidden and blocked for engineers. Keep every change surgical.

PART 2

Part 2 — Backend (Django)

Now build the Django backend that will replace the mocks. It serves the same JSON shapes the frontend already expects, adds JWT auth with roles, the admin management APIs, and the ML anomaly service.

Now build the Django backend that will replace the mocks. It serves the same JSON shapes the frontend already expects, adds JWT auth with roles, the admin management APIs, and the ML anomaly service.

1

STAGE 1

Project & Auth Foundation

Django project, PostgreSQL, custom User with roles, JWT, health check.

PASTE THIS INTO CLAUDE CODE

Start the Django backend for "PetroSense".
STACK: Python 3.11, Django + Django REST Framework, PostgreSQL, django-cors-headers, djangorestframework-simplejwt.

DO IN THIS STAGE ONLY:

1. Create project "petrosense" and app "core". Install the stack above.
2. Configure settings to read SECRET_KEY, DEBUG and the PostgreSQL connection from a .env file (use python-decouple or os.environ). Add DRF + JWT + CORS to INSTALLED_APPS/MIDDLEWARE. Allow the frontend origin http://localhost:5173.
3. Use a custom User model (AbstractUser) with a role field: 'admin' | 'engineer' (default 'engineer'). Set AUTH_USER_MODEL.
4. Configure DRF default auth = JWT, default permission = IsAuthenticated.
5. Add a /api/health/ endpoint returning {"status":"ok"} (public).
6. Give me the commands for makemigrations/migrate and creating a superuser (admin).

Explain the project layout and how to run the dev server.

2

STAGE 2

Models & Read APIs

Wells, readings, alerts + the read endpoints (camelCase to match the frontend) + seed data.

PASTE THIS INTO CLAUDE CODE

Add the data models and read APIs to PetroSense (Django + DRF).

1. Models in core/models.py:
 - Well: name, location, status (default 'normal'), created_at.
 - SensorReading: well (FK), timestamp, flow_rate, pressure, temperature, vibration, is_anomaly (bool).
 - Alert: well (FK), reading (FK), severity, reason (text), acknowledged (bool), created_at.
 (User already exists with role from Stage 1.)
2. DRF serializers. IMPORTANT: output camelCase JSON to match the frontend contract (wellId, flowRate, pressure, temperature, vibration, timestamp, isAnomaly, readingId, createdAt, acknowledged). Use a camelCase renderer or source= aliases.
3. ViewSets / endpoints:
 - GET /api/wells/ , GET /api/wells/{id}/ , GET /api/wells/{id}/readings/?limit=100 , GET /api/alerts/ , GET /api/stats/summary/ (counts by status + open alerts).
4. A management command seed_data that inserts 5 wells with realistic Egyptian field names and ~200 readings each (mostly normal, a few anomalies: flow down, pressure/temp/vibration up). Idempotent.
5. Register routers under /api/. Show sample curl for each endpoint.

3

STAGE 3

Roles & Admin APIs

JWT login, the IsAdmin permission, and the user + well management endpoints.

PASTE THIS INTO CLAUDE CODE

Add JWT auth, roles, and the ADMIN management APIs to PetroSense.

1. Auth endpoints (SimpleJWT):
 - POST /api/auth/login/ -> {access, refresh, user:{id,name,email,role}}
 - POST /api/auth/refresh/
 - GET /api/auth/me/ -> current user.
2. A custom permission IsAdmin (role == 'admin').
3. USER MANAGEMENT (admin only):
 - GET /api/admin/users/ list users
 - POST /api/admin/users/ create user (name, email, role, password -> hashed)
 - PATCH /api/admin/users/{id}/ update role / details
 - DELETE /api/admin/users/{id}/ delete user
 Protect ALL of these with IsAdmin.
4. WELL MANAGEMENT (admin only) – extend the wells viewset:
 - POST /api/wells/ create well (name, location) [IsAdmin]
 - DELETE /api/wells/{id}/ delete well [IsAdmin]
 Keep GET /api/wells/ readable by any authenticated user.
5. Return clean 401/403 errors. Show curl to log in as admin, create an engineer, create a well, and prove an engineer token is REJECTED (403) on admin routes.

ML Anomaly Service

Train the Random Forest and expose `/api/predict/` with an explainable reason.

PASTE THIS INTO CLAUDE CODE

Add the ML anomaly service to PetroSense (Django).

1. `ml/train.py`: load the labelled oil-well CSV with pandas; features [flow_rate, pressure, temperature, vibration, pressure_temp_ratio, flow_vibration_ratio], target is_anomaly; stratified split; train a `RandomForestClassifier(class_weight="balanced")`; print accuracy/precision/recall/F1 + feature_importances_; save to `ml/model.pkl` with joblib.
2. `core/ml_service.py`: load `model.pkl` once at startup; `predict(reading) -> {is_anomaly, probability, top_features}` where `top_features` combines `feature_importances_` with each input's deviation from normal.
3. `POST /api/predict/` (authenticated): body {flow_rate, pressure, temperature, vibration}; compute ratios server-side; return prediction + a human-readable reason like "Vibration and pressure abnormally high".

Show training output and a sample `/api/predict/` request + response.

PART 3

Part 3 — Database (PostgreSQL)

The database is four tables the whole product sits on. Keep them simple and correct; the relationships are what let an alert point back to the exact reading that caused it.

The database is four tables the whole product sits on. Keep them simple and correct; the relationships are what let an alert point back to the exact reading that caused it.

1

STAGE 1

Schema & Migrations

The 4 tables, relationships, indexes, and the Django migrations.

PASTE THIS INTO CLAUDE CODE

```
Design and document the PetroSense PostgreSQL schema (Django migrations).  
Produce the final schema with 4 tables and their relationships:  
- users(id, name, email UNIQUE, password[hashed], role['admin'|'engineer'], created_at)  
- wells(id, name, location, status['normal'|'warning'|'critical'], created_at)  
- sensor_readings(id, well_id FK->wells, timestamp, flow_rate, pressure,  
  temperature, vibration, is_anomaly)  
- alerts(id, well_id FK->wells, reading_id FK->sensor_readings, severity, reason,  
  acknowledged, created_at)  
Relationships: one well -> many readings, one well -> many alerts, each alert -> one  
reading. Add sensible indexes (well_id, timestamp, severity). Generate the Django  
migrations, show the final migration files, and give an ER description in words.  
Explain why each index exists.
```

PART 4

Part 4 — Connecting the API (step by step)

Finally, connect the two — one step at a time, touching ONLY `/src/services/api.ts` each time, so no screen changes and nothing breaks. Do these in order after the backend runs.

Finally, connect the two — one step at a time, touching ONLY `/src/services/api.ts` each time, so no screen changes and nothing breaks. Do these in order after the backend runs.

1

STAGE 1

Auth wiring

Axios instance + real login/refresh/me, token from the store.

PASTE THIS INTO CLAUDE CODE

```
Connect the PetroSense React frontend to the real Django API. Work ONLY inside
/src/services/api.ts plus a tiny axios setup – do not touch screens.
1. Create /src/lib/axios.ts: an Axios instance with baseURL from
import.meta.env.VITE_API_URL, plus an interceptor that attaches
Authorization: Bearer <token> from the Zustand auth store.
2. In api.ts, replace the MOCK body of login/getCurrentUser with real calls to
/api/auth/login/ and /api/auth/me/. Store access token in the auth store.
3. Add a 401 response interceptor that tries /api/auth/refresh/ once, else logs out.
Keep the SAME function signatures and the SAME return shapes the screens already use,
so no screen changes. Show me the .env variable to set.
```

2**STAGE 2**
Data reads

Swap wells/readings/alerts/stats from mock to the real API.

PASTE THIS INTO CLAUDE CODE

```
Switch the PetroSense data reads from mock to the real Django API.  
Work ONLY in /src/services/api.ts.  
Replace the mock bodies of getWells, getWell, getSensorHistory, getAlerts, getKpis  
with real Axios calls to /api/wells/, /api/wells/{id}/, /api/wells/{id}/readings/,  
/api/alerts/, /api/stats/summary/. Keep identical return shapes (the backend already  
sends camelCase). Verify the Overview, Well Detail and Alerts screens render real data  
with zero changes to the screens themselves. Keep the live-chart interval working by  
polling readings. Surgical – api.ts only.
```

3**STAGE 3**
Admin actions + test

Wire user/well management + predict, then an end-to-end role test.

PASTE THIS INTO CLAUDE CODE

```
Wire the ADMIN actions to the real Django API. Work ONLY in /src/services/api.ts.  
Replace mock bodies of getUsers, createUser, deleteUser with /api/admin/users/ calls,  
and createWell, deleteWell with POST/DELETE /api/wells/. Keep the same signatures and  
shapes so the admin screens are untouched. Confirm: an engineer token gets 403 on these  
(the UI already hides the nav, but the API must enforce it too). Add a POST /api/predict/  
helper. Surgical – api.ts only. Then give me a short end-to-end test checklist  
(log in as admin, add a well, add an engineer, log in as that engineer, confirm no admin  
nav and 403 on admin routes).
```